

Code templates & reference notes

Track 16.7 — Claude Code and the Agent SDK · 16-ai-tech-stack/tracks/claude-code/code/claude-md-template.md · 16-ai-tech-stack/tracks/claude-code/code/example-skill/README.md · 16-ai-tech-stack/tracks/claude-code/code/example-skill/SKILL.md · 16-ai-tech-stack/tracks/claude-code/code/hooks-README.md

Annotated `CLAUDE.md` template — for an AI/ML project

Copy the fenced block below to `CLAUDE.md` at your repo root (or `.claude/CLAUDE.md`), then delete every line that doesn't pass the test in §0. The commentary after the block explains *why each section earns its place* — a section that can't answer "would removing this cause Claude to make a mistake?" should not be there.

Generate a first draft with `/init` rather than typing this from scratch. Then prune. `/init` is good at build commands and layout, and has no idea which of your conventions are load-bearing.

0. The test every line must pass

Would removing this line cause Claude to make a mistake? If no, cut it.

Target: **under 200 lines**. The docs are explicit that longer files consume more context *and reduce adherence* — a bloated `CLAUDE.md` makes Claude follow your rules **less** reliably, because the important ones are buried. It is loaded into every request forever, so a line here is the most expensive place in the repo to put a sentence.

The template

```
# <project-name>

Recommendation service: FastAPI + a fine-tuned ranker, served on GPU, trained offline in `training/`.
Python 3.12, `uv` for deps.

## Commands
- Install: `uv sync --all-extras`
- Test (all): `uv run pytest`
- Test (one): `uv run pytest tests/test_ranker.py::test_scores_are_calibrated -x`
- Lint + format: `uv run ruff check --fix . && uv run ruff format .`
- Types: `uv run mypy src/`
- Serve locally: `uv run uvicorn app.main:app --reload --port 8080`
- Train a smoke model (~90s, CPU): `uv run python training/train.py --config configs/smoke.yaml`

## Layout
- `src/app/` – FastAPI service. Route handlers stay thin; logic lives in `src/app/services/`.
- `src/ranker/` – model code. Pure functions, no I/O, no config reads.
- `training/` – offline jobs. May import `src/ranker`; must never import `src/app`.
- `configs/` – Hydra YAML. Every experiment is a config, never a code edit.
- `data/` – gitignored. Never commit anything under it.

## Conventions
- Type hints on every public function. `mypy --strict` is clean; keep it that way.
- Pydantic v2 models for every API boundary and every config object.
- Use `structlog`, not `print` and not `stdlib logging`.
- Random seeds come from `src/ranker/seed.py::set_seed`. Never call `np.random.seed` directly.
- New dependency: add to `pyproject.toml` and run `uv lock`. Do not `pip install`.

## Things that will bite you
- `src/ranker/features.py` builds features from an event stream. Any feature must use a timestamp
  strictly
  earlier than the prediction timestamp, or it is leakage. This has shipped to prod twice. Check it.
- The GPU box is Ampere; `torch.compile` with `mode="max-autotune"` hangs there. Use the default mode.
- `tests/integration/` needs a local Redis (`docker compose up -d redis`). Skip them with `-m "not
  integration"`.
- IMPORTANT: model artifacts in `artifacts/` are immutable. Train a new version; never overwrite one.

## Workflow
- Branch naming: `feat/<ticket>-short-slug`, `fix/<ticket>-short-slug`.
- Run lint, types, and the unit tests before you say a change is done. Show me the output, not a summary.
- Never commit or push unless I ask.
- For changes under `src/ranker/`, use plan mode first – a change there moves an offline metric.

## Compact instructions
When compacting, preserve: the failing test output, any metric numbers we discussed, and the current
hypothesis about the bug. Drop file listings and successful tool output.

@docs/architecture.md
```

Why each section earns its place

<project-name> — the two-line orientation

Claude opens a fresh context window every session. Two sentences of "what is this and what stack" saves it three or four exploratory file reads on *every* session, which is the cheapest trade in the entire file. Keep it to what it does, the stack, and the runtime — not a pitch.

Commands — the single highest-value section

This is the canonical example of "bash commands Claude can't guess." If your test command is `pytest`, you don't need this section. If it's `uv run pytest` behind a `--all-extras sync`, or `make test-unit ENV=ci`, Claude will guess wrong, the guess will fail, and it will spend turns recovering.

Two details that pay for themselves:

- **Include the single-test form.** The docs recommend preferring one test over the whole suite for performance. If Claude knows the incantation it will use it; if not it runs your 8-minute suite to check a one-line change.
- **Note anything slow or expensive** (~90s, needs GPU) so Claude can decide whether to run it or ask.

Layout — only the parts that aren't obvious

Not a file-by-file tour; Claude can read a directory listing. What it cannot infer is **the dependency rules**: "`training/` may import `src/ranker` but must never import `src/app`" is an architectural constraint that exists only in your head and in a code review you're tired of writing. Same for "route handlers stay thin" — that is a decision, not a discoverable fact.

If this section grows past ~10 lines, it belongs in a `.claude/rules/` file scoped with `paths:` frontmatter so it only loads when Claude opens files in that area.

Conventions — only the deltas from the default

Claude already writes type hints and already knows PEP 8. What it can't know is that you standardised on `structlog`, that seeds go through one helper, or that your dependency workflow is `uv lock` rather than `pip`. Every line here should be a **delta from what a competent engineer would do by default**. "Write clean code" and "use meaningful variable names" are the canonical wasted lines.

Things that will bite you — the section that actually changes outcomes

This is where a `CLAUDE.md` stops being documentation and starts being useful. Every entry is a real bug that has happened, with the specific mechanism named. The leakage line is worth more than the other three sections combined, because the mistake is invisible in tests and expensive in production (see `../../../01-ml-foundations/`).

Populate it from: code review comments you've written twice, incidents, and corrections you keep typing into chat. That's the documented trigger — "*Claude makes the same mistake a second time*" — and it applies to humans too.

Note the single **IMPORTANT:** marker. The docs say emphasis works **when it is rare**: "If you emphasize many lines, none of them stands out." One **IMPORTANT** per file, maybe two. Zero is also fine.

Workflow — repository etiquette

Branch naming and "never commit unless I ask" are cheap to state and annoying to undo. "**Show me the output, not a summary**" is the highest-leverage line in the section: it converts Claude's self-report ("tests

pass") into evidence you can check, which is the documented cure for the trust-then-verify gap.

The last line — plan mode for `src/ranker/` — routes a class of risky change into the slower, safer path without you remembering to ask.

Compact instructions — the section almost nobody writes

A documented feature: a "Compact instructions" section in `CLAUDE.md` steers what survives summarization when the window fills. Long sessions are where context is silently lost, and losing the failing test output or the metric you were chasing costs you a re-run. Four lines here save real money.

@docs/architecture.md — the import, used sparingly

`@path` imports are expanded **at launch**, so an imported file costs exactly what pasting it in would cost. Use an import when the content is genuinely needed every session and is maintained elsewhere (an `AGENTS.md` you share with another tool, a design doc the team edits). Do **not** use it to pretend your 600-line file is small. If the content is only needed sometimes, it is a **skill**, not an import.

What deliberately isn't here, and where it went instead

Content	Why not in <code>CLAUDE.md</code>	Where it goes
The release checklist	needed occasionally, ~40 lines	a skill: <code>.claude/skills/release/SKILL.md</code>
Frontend style rules in a monorepo	only relevant under <code>web/</code>	<code>.claude/rules/frontend.md</code> with paths: <code>["web/**"]</code>
"Never read <code>.env</code> "	a request, not enforcement	<code>permissions.deny: ["Read(./env)"]</code> + a <code>PreToolUse</code> hook
"Always run the formatter after editing"	Claude may forget; a hook cannot	a <code>PostToolUse</code> hook on <code>Edit\ Write</code>
Full API reference for an internal service	huge, changes weekly	a skill, or an MCP server
Your personal editor preferences	not the team's business	<code>~/claude/CLAUDE.md</code>
Your sandbox URLs and test credentials	must not be committed	<code>CLAUDE.local.md</code> (gitignored)

That table is the real lesson of this file. **Most of what people put in `CLAUDE.md` belongs somewhere cheaper.**

Maintenance

- Run `/context` and confirm the file appears under **Memory files**. If it isn't there, nothing else matters.
- Re-read it monthly. Delete rules for problems that no longer exist. Conflicting rules make Claude pick one arbitrarily, and a stale rule is worse than no rule.
- If Claude keeps violating one specific rule, the file is probably too long — prune before you add emphasis.

- If Claude asks you something this file already answers, the file is too long or the answer is buried.
- Every time you catch yourself typing the same correction twice: that's a new line here, a new hook, or a new skill. Decide which, deliberately. Guidance → here. Guarantee → hook. Procedure → skill.

The `leakage-check` skill — install notes and design rationale

Install

```
# project-scoped (committed, shared with your team)
mkdir -p .claude/skills/leakage-check
cp SKILL.md .claude/skills/leakage-check/

# or personal (every project on this machine)
mkdir -p ~/.claude/skills/leakage-check
cp SKILL.md ~/.claude/skills/leakage-check/
```

Then in a session: `/reload-skills`, check it appears in `/skills`, and run `/leakage-check` or `/leakage-check src/ranker/features.py`.

The command name comes from the directory, not the frontmatter. For personal and project skills, `name:` is only the display label in listings; you invoke `<directory-name>`. So copy `SKILL.md` into a directory called `leakage-check`, not into one called `example-skill`. (Plugin skills are the exception: there `name` *does* set the last segment of `/plugin-name:skill-name`.)

Why each frontmatter field is there

Every field is optional; only `description` is recommended. These are the ones that earn their place here:

Field	Why
<code>name</code>	Display label in <code>/skills</code> and <code>/context</code> . Cosmetic for a project skill, but keeps listings readable.
<code>description</code>	The load-bearing field. Claude reads this to decide whether the skill is relevant. Put the key use case first — <code>description</code> + <code>when_to_use</code> are truncated at 1,536 characters in the skill listing.
<code>when_to_use</code>	Trigger phrases and example requests. Appended to <code>description</code> in the listing, so it counts toward the same cap. Use it when the description alone leaves the trigger ambiguous.
<code>argument-hint</code>	Autocomplete hint (<code>[file-or-directory]</code>) so you can see what it takes without opening the file.
<code>allowed-tools</code>	Pre-approves <code>Read</code> , <code>Grep</code> , <code>Glob</code> for the turn that invokes the skill, so an audit doesn't generate three permission prompts. The grant clears when you send your next message.
<code>disallow-d-tools</code>	Removes <code>Write</code> and <code>Edit</code> from Claude's pool while the skill is active. This is the difference between "the instructions say report only" and report only . Also clears at your next message.

Fields deliberately **not** used, and why:

- `disable-model-invocation: true` — would stop Claude loading this automatically. Here, auto-loading is the point: you *want* it to fire when you say "review my feature code" without remembering the command name.
- `context: fork` — would run it in a subagent. Good for long audits that would flood your window; overkill for a diff-sized review, and you lose the ability to ask a follow-up question against the same context.
- `model / effort` — the session defaults are fine. Override these only when a skill genuinely needs more reasoning than the rest of your session, because the override applies to the whole turn.
- `paths` — would restrict auto-activation to matching files. Worth adding in a monorepo (`paths: ["src/ranker/**", "training/**"]`); left off here so the example works anywhere.

The body: two mechanics worth copying

Dynamic context injection. The line

```
!`git diff HEAD --stat`
```

is executed by Claude Code *before* Claude sees the skill content, and replaced by its output. So the instructions arrive with the current diff already inlined — the skill is grounded in your real working tree rather than in whatever files happen to be open. Use `!`cmd`` for cheap, bounded commands; a command that prints megabytes will simply spend your context.

\$ARGUMENTS. Everything you type after `/leakage-check` lands here. When no placeholder consumes them, Claude Code appends them as `ARGUMENTS: <value>` instead. Named arguments are available too via the arguments frontmatter list (`arguments: [target, severity] → $target, $severity`).

Keep the body short. Once a skill loads, its content stays in context across turns — every line is a recurring token cost. State what to do; don't narrate why. This one is ~35 lines and that is close to the ceiling for something you invoke often.

When a skill beats a slash command

Short answer: **custom commands have been merged into skills.** A file at `.claude/commands/deploy.md` and a skill at `.claude/skills/deploy/SKILL.md` both give you `/deploy` and both still work. Existing `commands/` files are not deprecated. So the real question is what a skill directory adds over a single command file:

	<code>.claude/commands/x.md</code>	<code>.claude/skills/x/SKILL.md</code>
Invoke with <code>/x</code>	yes	yes
Frontmatter	most fields, but not <code>name</code> or <code>paths</code>	all fields
Supporting files (scripts, templates, reference docs)	no — one file	yes — it's a directory
Claude can load it automatically when relevant	no	yes, unless <code>disable-model-invocation: true</code>
Run in a forked subagent (<code>context: fork</code>)	no	yes
Ships in a plugin with <code>agents/hooks/MCP</code>	no	yes

Use a skill when:

- You want Claude to reach for it *without being asked* — the auto-invocation path is the biggest single difference, and it's what turns a command you forget into a habit that fires on its own. (This skill is the example: you say "review my feature changes", it loads.)
- It needs supporting files: a checklist, a schema, a script the skill runs via `${CLAUDE_SKILL_DIR}`.
- It should run in isolated context (`context: fork`) because it reads a lot and returns a little.
- It's going into a plugin for the team.
- It's reference material rather than an action — an API style guide Claude should apply while working. A command file can't do this at all, because nothing loads it unless you type it.

A command file is still fine when it's a three-line prompt you type yourself and nothing more. Prefer a skill for new work anyway; the directory costs you one `mkdir` and leaves every door open.

And the comparison that actually matters — skill vs the alternatives:

- vs **CLAUDE.md**: `CLAUDE.md` loads every session forever. A skill's body loads only when used. Anything that isn't needed in *every* session belongs in a skill. This is the main reason to reach for one.
- vs **subagent**: a skill runs in your main context and shares it; a subagent has its own window and returns only a summary. Need isolation? Subagent (or `context: fork`).
- vs **hook**: a skill is instructions Claude interprets — the outcome can vary. A hook always fires on its event. Guidance goes in a skill; **guarantees go in a hook**.

SKILL

name: leakage-check description: Audits a diff or a named file for data-leakage and reproducibility bugs in ML feature or training code. Use when the user asks to review ML changes, mentions feature engineering, train/test splits, seeds, or asks "is this leaking?". when_to_use: Invoke before merging anything under a feature-engineering, dataset, or training directory. Trigger phrases: "review my features", "check for leakage",

"is this split honest", "audit this training script". argument-hint: [file-or-directory] allowed-tools: Read, Grep, Glob disallowed-tools: Write, Edit

Changes under review

```
!git diff HEAD --stat
```

Scope

Audit \$ARGUMENTS if given; otherwise audit the changed files shown above. Read only — do not edit anything.

Checks

Work through these in order. For each finding, report `file:line`, the mechanism, and the concrete fix.

1. **Temporal leakage.** Any feature computed from a row whose timestamp is $\geq$ the prediction timestamp. Look for joins, rolling windows, groupby aggregations and `shift()` calls with no time bound.
2. **Fit-before-split.** Scalars, imputers, encoders or vocabularies fitted on the full dataset rather than inside the training fold. Grep for `fit( / fit_transform(` outside a `Pipeline` or a CV loop.
3. **Target-derived features.** Any column computed from, or only knowable after, the label. Names containing `_reason`, `_outcome`, `closed_`, `final_` deserve a second look.
4. **Split integrity.** Random splits on time-ordered data; rows sharing an entity (user, patient, session) landing on both sides; duplicate rows spanning the split.
5. **Non-determinism.** Seeds not set, or set per-library rather than through one helper; `set()` or `dict` iteration order feeding into data order; `num_workers > 0` without a worker seed.
6. **Test-set contamination.** The test set read more than once, or used for early stopping or threshold tuning.

Output

- If clean: one line saying so, plus the highest-risk area you would watch next.
- Otherwise: a markdown table `severity | file:line | mechanism | fix`, ordered high to low severity.
- End with one sentence: what would have to be true for this code to be safe.

Never modify files. Report only.

`example-hooks-settings.json` — what every entry does, and why

This file is a working `.claude/settings.json`. It does three genuinely useful things:

1. **Blocks edits to protected paths** — deterministically, not by asking nicely.
2. **Blocks `git push --force` on protected branches** before the command runs.
3. **Runs the formatter after every write**, so Claude never hands you unformatted code.

Plus a `SessionStart` line that prints the current branch into context, because the single most common "why did it edit the wrong thing" is a session on a branch you forgot you were on.

Install

```
cp example-hooks-settings.json <your-repo>/claude/settings.json # merge if one exists
mkdir -p <your-repo>/claude/hooks
# then create the three scripts below and: chmod +x .claude/hooks/*.sh
```

Verify with `/hooks` in a session, and `/doctor` if something doesn't fire. `settings.json` is read at session start — restart after editing it. Hooks in a project settings file require you to have trusted the workspace.

Read the hooks you install. A hook is a shell command Claude Code runs on your machine with your credentials, on someone else's trigger. Treat a `settings.json` arriving in a pull request exactly as you would treat a new Makefile target: read it before you run it. The docs carry the same disclaimer.

The structure: three levels of nesting

Every hook config is the same shape, and if you can recite it you can write any hook:

```
hooks
├─ <HookEvent>           e.g. "PreToolUse"           - the lifecycle point
├─ [ matcher group ]     e.g. {"matcher": "Edit"}     - which occurrences it fires on
└─ hooks: [ ... ]       the handlers that run        - command / http / mcp_tool / prompt / agent
```

All matching handlers run **in parallel**. A handler defined identically in two settings files runs once.

Entry by entry

1. PreToolUse on Write|Edit|NotebookEdit → protect-paths.sh

Why PreToolUse: it is the only tool event that can *stop* a call. `PostToolUse` fires after the damage.

Why this matcher: a matcher made only of letters, digits, `_`, `-`, spaces, `,` and `|` is treated as a list of **exact** tool names. `Edit|Write` matches those two tools and nothing else. Anything else in the string makes it an **unanchored JavaScript regex** — which is why `Edit.*` would also match `NotebookEdit`. Here we list all three explicitly, which is clearer than relying on a regex to be sloppy in a convenient direction.

Why NotebookEdit is in the list: Read deny rules cover `Edit` and `Write` on the same path but the docs note `NotebookEdit` isn't covered, so a notebook is a real bypass. Name it.

Why exec form (`"args": []`): a command hook runs in exec form whenever `args` is present — `command` is resolved as an executable and spawned directly, no shell, so `${CLAUDE_PROJECT_DIR}` is substituted as one literal argument. In shell form a path containing a space would be re-tokenized. The docs recommend exec form for any hook referencing a path placeholder. (On Windows, exec form needs a real executable — `.cmd/.bat` shims won't spawn.)

Why \${CLAUDE_PROJECT_DIR}: it is the project root where the session started, regardless of Claude's current directory. It even stays put when Claude enters a worktree — the worktree path arrives as `cwd` in the hook's

JSON input instead.

The script:

```
#!/usr/bin/env bash
# .claude/hooks/protect-paths.sh
# PreToolUse on Write|Edit|NotebookEdit. Exit 2 = block.
set -euo pipefail
input=$(cat)                                # event JSON arrives on stdin
file_path=$(printf '%s' "$input" | jq -r '.tool_input.file_path // ""')
file_path="${file_path//\\//}"              # normalize Windows separators

case "$file_path" in
  */.github/workflows/*|*/migrations/*|*/.claude/settings.json|*.lock|*/go.sum|*/uv.lock)
    echo "Blocked: $file_path is protected. Ask a human to change it." >&2
    exit 2
  ;;
esac
exit 0
```

Why exit 2 and not exit 1. On events that can block, exit 2 blocks — and your **stderr** becomes the reason Claude reads, so it can adapt instead of retrying blindly. Exit 1 is a *non-blocking* error: the transcript shows a hook-error notice and **the tool call proceeds anyway**. This is the opposite of Unix instinct and it is the single most common hook bug. Exit 2 also beats JSON: even a `permissionDecision: "allow"` on stdout cannot override it.

Why `tool_input.file_path` is safe to match on: for Write, Edit and Read, Claude Code expands `~` and relative paths *before* hooks run, so the path is always absolute. A hook can't be dodged with `./foo/../../env`. The `${file_path//\\//}` line handles Windows, where the path arrives with backslashes even under Git Bash — a `/migrations/` check would silently never match otherwise, and the call would sail through.

The structured-JSON alternative. Instead of exit 2 you can exit 0 and print:

```
{ "hookSpecificOutput": { "hookEventName": "PreToolUse",
  "permissionDecision": "deny", "permissionDecisionReason": "Protected path" } }
```

`permissionDecision` takes `allow`, `deny`, `ask`, or `defer`; when several `PreToolUse` hooks disagree, precedence is `deny > defer > ask > allow`. Use JSON when you want `ask` (send it to the human) or `updatedInput` (rewrite the call). Use exit 2 when the answer is simply no — it's three lines of bash instead of a JSON template.

2. PreToolUse on Bash with `if: "Bash(git push *)"` → `block-force-push.sh`

Why the `if` field: the `matcher` filters on the tool *name* only, so `"Bash"` fires on every shell command. `if` takes exactly one **permission-rule** pattern and matches the tool name *and arguments together*, so the script only runs for pushes. That keeps a hook off the hot path of every `ls`.

Behaviour worth knowing: leading `VAR=value` assignments are stripped before matching; each subcommand of a `&& b` is checked; commands inside `$()` and backticks are checked. And when Claude Code can't tell what a command expands to (`$TOOL git push`), it runs your hook anyway — `if` is deliberately **fail-open on**

ambiguity, which is why the docs say to use the permission system, not `if`, for a hard allow/deny. There is no `&&/||` in `if`: one rule per handler, add another handler for another condition.

```
#!/usr/bin/env bash
# .claude/hooks/block-force-push.sh
set -euo pipefail
input=$(cat)
cmd=$(printf '%s' "$input" | jq -r '.tool_input.command // ""')
branch=$(git rev-parse --abbrev-ref HEAD 2>/dev/null || echo "")

if printf '%s' "$cmd" | grep -Eq -- '--force(-with-lease)?|(^| )-f( |$)'; then
  case "$branch" in
    main|master|release/*)
      echo "Blocked: force-push to '$branch'. Push to a feature branch and open a PR." >&2
      exit 2
    ;;
  esac
fi
exit 0
```

Be honest about the limit. This inspects the command text Claude wrote. It does not stop `/usr/bin/git push --force` or `sh -c 'git push -f'` — the docs say the same about `Bash(...)` permission rules. Command-text inspection is a guardrail against Claude doing something dumb, **not** a security boundary against an adversary. For a real boundary use [sandboxing](#), a container, or a server-side branch protection rule. Defence in depth: this hook plus GitHub branch protection.

3. PostToolUse on Write|Edit → `format-after-write.sh`

Why PostToolUse: the file exists now, so we can act on it. The input carries both `tool_input` (arguments) and `tool_response` (result). Formatting is the canonical hook: it must happen *every* time, and it needs no reasoning — the docs' own rule is that deterministic, no-thinking actions are hooks, not instructions.

```
#!/usr/bin/env bash
# .claude/hooks/format-after-write.sh
set -euo pipefail
input=$(cat)
file_path=$(printf '%s' "$input" | jq -r '.tool_input.file_path // ""')
file_path="${file_path//\\//}"
[ -f "$file_path" ] || exit 0

case "$file_path" in
  *.py)          command -v ruff >/dev/null && ruff format -q "$file_path" ;;
  *.ts|*.tsx|*.js|*.jsx|*.json|*.md)
    command -v prettier >/dev/null && prettier --write --log-level warn "$file_path" ;;
  *.go)          command -v gofmt >/dev/null && gofmt -w "$file_path" ;;
  *.rs)          command -v rustfmt >/dev/null && rustfmt --edition 2021 "$file_path" ;;
esac
exit 0
```

Always exit 0 from a formatter. A `PostToolUse` hook that exits 2 feeds a blocking error back to Claude, which will then try to "fix" it — and a missing `prettier` is not something Claude can fix. The `command -v` guards keep a teammate without your toolchain from getting an error on every edit. The `timeout: 60` is

generous because a cold `prettier` on a large file is slow; the default for command hooks is 600 seconds, which is far too long to sit behind a formatter.

To surface a warning to Claude instead of blocking, print JSON with `additionalContext` (or use `systemMessage`). Plain `stderr` from a hook that exits 0 goes to the debug log only — Claude never sees it.

4. `SessionStart` with matcher `startup|resume`

The matcher on `SessionStart` filters **how the session started** — `startup`, `resume`, `clear`, `compact` are the documented values. This one fires on a new or resumed session but not after a `/clear`, where you already know where you are.

`SessionStart` is one of only four events (with `UserPromptSubmit`, `UserPromptExpansion` and `PostModelSwitch`) where **plain stdout is added to Claude's context**. That makes it the cheap way to inject dynamic session facts: current branch, deploy target, whether the local stack is up, today's on-call. Keep it to a line or two — it is paid for on every request afterwards.

This entry is in shell form (no `args`) because it uses a pipeline and command substitution, which need a shell. That's the rule: **exec form for scripts and path placeholders, shell form when you need shell features**.

The permissions block: why both layers

The `permissions` object at the top isn't decoration. Hooks and permission rules do different jobs:

	<code>permissions.deny</code>	<code>PreToolUse</code> hook
Evaluated	before hooks in the rule chain; deny wins over ask and allow	first in the flow; can deny outright
Expressiveness	path/command patterns	arbitrary code — branch state, file contents, time of day, your API
Covers Bash file commands	yes: Read deny also covers <code>cat</code> , <code>head</code> , <code>sed</code> , and <code>></code> redirect targets	only what you inspect
Cost to maintain	a line of JSON	a script you own and must keep working
Fails how	closed	if the script is missing or non-executable, it's a non-blocking error — the action proceeds

So: **static, expressible-as-a-pattern** → **permission rule**. **Conditional, stateful, needs logic** → **hook**. Use both for anything that matters, because the hook can break and the rule cannot.

Note also `"ask": ["Bash(git push *)"]` sitting next to the force-push hook. Evaluation order is deny → ask → allow, first match wins, and specificity does not change the order — a broad deny will swallow a narrow allow, so don't try to write allowlist exceptions inside a deny.

Debugging

Symptom	Check
Hook never fires	<code>/hooks</code> shows what's registered; <code>settings.json</code> is read at session start — restart
Fires but doesn't block	you exited 1, not 2. Only exit 2 blocks
"hook error" in the transcript	script missing, not <code>chmod +x</code> , or stdout that looks like broken JSON
Fires on the wrong tool	your matcher contained a non-word character and became an unanchored regex
Works on macOS, not Windows	backslash separators in <code>file_path</code> ; normalize before comparing
Everything is weird	<code>claude --debug</code> or <code>/doctor</code> ; <code>--safe-mode</code> starts with all customizations off

Turn everything off for one run with `--settings '{"disableAllHooks": true}'`. Note that setting it in *user* settings isn't enough — project settings take precedence and can set it back to `false`.